

AN-DNA

A Deterministic Trust Verification Framework for Distributed Systems

Post-Quantum-Oriented Architecture for Replay-Resistant Authentication, Canonical Transcript Integrity, and Public Verification

Darrell Morris Jr.

Version: vNext-Phase1-R1 | May 2026 | Independent Research

Research Scope Notice

This is an experimental research implementation, not a production security standard. This manuscript documents the current implementation scope, verifier architecture, known limitations — including formal proof requirements and external review needs — and clearly delineated future work. The project is intentionally constrained: deterministic reproducibility and bounded verification guarantees, not generalized identity or behavioral inference.

Abstract

The accelerating degradation of digital trust presents a growing challenge for distributed systems, infrastructure authentication, software provenance, and machine-mediated communication. Traditional trust assumptions — persistent credentials, static identity artifacts, centralized verification, and long-lived cryptographic validity — are increasingly vulnerable to replay attacks, implementation drift, synthetic impersonation, and long-term cryptanalytic uncertainty.

This manuscript describes AN-DNA (Adaptive Network DNA Architecture), a deterministic trust verification framework and its accompanying reference implementation. The framework is scoped narrowly as a deterministic trust substrate for validating integrity, authenticity, and authorization continuity across heterogeneous digital environments.

The system is organized around five operational principles:

- Deterministic transcript construction
- Canonical payload serialization with compile-time enforcement

- Replay-resistant epoch-bound verification semantics
- Public-verifier post-quantum signature architecture
- Minimum-necessary trust disclosure

The reference implementation is a production Rust workspace (andna-core) with a Python verification layer. The Rust crate provides ML-DSA-44 signature verification via FIPS 204-aligned liboqs bindings. Three Known Answer Test (KAT) vectors are validated byte-for-byte between Rust and Python. Compile-time assertions enforce the canonical payload layout across all language bindings.

1. Introduction

Modern distributed systems operate in environments where trust assumptions are degrading faster than conventional verification models can adapt. Authentication systems have historically depended on assumptions now under simultaneous pressure:

- that communication endpoints are authentic
- that credentials remain private
- that signatures remain computationally secure indefinitely
- that implementations interpret data identically
- that centralized verification authorities remain trustworthy

These assumptions are challenged by large-scale credential theft, replay-based attacks, distributed infrastructure complexity, software supply-chain compromise, implementation drift, and the transition toward post-quantum cryptographic environments.

In practice, many failures in modern trust systems do not emerge from failures in cryptographic mathematics, but from inconsistencies in payload interpretation, serialization behavior, verifier implementation, and state synchronization.

AN-DNA is designed around a narrow but defensible objective: enable reproducible, replay-resistant, publicly verifiable integrity and authorization continuity across distributed systems.

2. Design Philosophy

2.1 Trust as Deterministic Continuity

Traditional authentication models trust as possession of a password, a persistent credential, or a long-lived cryptographic key. AN-DNA models trust as the deterministic continuity of a bounded verification state.

The framework does not attempt to prove generalized truth, intent, or behavioral legitimacy. It proves:

- integrity of a canonical artifact

- continuity of authorized transitions within epoch boundaries
- replay resistance through temporal binding
- reproducibility of verification across independent implementations

2.2 Minimum Necessary Trust

Systems should disclose only the minimum required information, to the minimum required verifier, for the minimum required duration, under the minimum required scope. This principle shapes payload construction, transcript formation, and replay invalidation semantics throughout the implementation.

2.3 Verifier / Prover Separation

The andna-core implementation is a verifier reference. It receives a proof frame from a prover and determines whether that frame satisfies all verification directives. The construction of epoch public keys (T_E) from device polynomial identity — the prover-side D0 epoch-witness bridge — is defined by domain separation constants in the contracts crate (EPOCH_SEED_DOMAIN, MLDSA_SEED_DOMAIN) and described in the companion protocol specification, but its implementation is outside the scope of this verifier codebase. This separation mirrors the public-verifier semantics described in Section 6.

3. Canonical Payload Construction

3.1 Problem Statement

Distributed verification systems frequently fail due to implementation divergence rather than cryptographic failure. Field ordering inconsistencies, language-dependent serialization, endianness mismatches, and undocumented transformation behavior create conditions where one verifier accepts a payload and another rejects it, destroying deterministic consensus and auditability.

3.2 Canonical Payload Semantics

AN-DNA addresses this through strict canonical payload construction enforced at compile time. The contracts crate (andna-contracts) is the single source of truth for all constants, field offsets, and size invariants. Every language binding derives from this crate or its generated artifacts. A change to any field offset requires a new specification version, a version byte increment, and KAT regeneration.

Property	Enforcement	Consequence of Violation
Fixed field ordering	Offset constants, compile-time assertions	Build failure
Deterministic serialization	SHAKE256 throughout; no locale-sensitive ops	KAT mismatch
Explicit byte-length constraints	const _: () = assert!(...) at compile time	Build failure

Language-independent encoding	Rust + Python KAT parity verified byte-for-byte	Test failure
No undefined byte regions	All 274 bytes of mu_pre are assigned fields	Layout audit failure

3.3 The mu_pre Canonical Payload (274 Bytes)

The central canonical artifact is mu_pre, a 274-byte deterministic buffer from which the signing message μ is derived as: $\mu = \text{SHAKE256}(\text{mu_pre}, 64)$.

Offset	Bytes	Field	Value / Derivation
0x0000	64	pk_hash	SHAKE256(T_E, 64)
0x0040	9	domain_sep	"ANDNAAUTH" (ASCII, no NUL)
0x0049	1	version	0x01
0x004A	32	device_id32	SHAKE256(T_E.device_id16, 32)
0x006A	8	epoch	uint64 little-endian
0x0072	32	sid	Session identifier
0x0092	32	N_d	Device nonce
0x00B2	32	N_s	Server nonce
0x00D2	32	ctx_hash	SHAKE256(context, 32)
0x00F2	32	policy_hash	Phase 1: zeroed (reserved for Phase 2)

Note: The policy_hash field at offset 0x00F2 is reserved and zeroed in Phase 1. It is allocated and offset-locked so that Phase 2 activation requires only populating an existing field, not a layout change.

3.4 Compile-Time Offset Enforcement

Every offset is verified at compile time via Rust const assertions. A build that silently shifts any field does not compile, eliminating the entire class of serialization drift failures:

```
const _: () = assert!(MU_PRE_DOMAIN_SEP_OFF == MU_PRE_PK_HASH_LEN);
const _: () = assert!(MU_PRE_VERSION_OFF == MU_PRE_DOMAIN_SEP_OFF +
MU_PRE_DOMAIN_SEP_LEN);
// ... full chain covers all 274 bytes ...
const _: () = assert!(MU_PRE_POLICY_HASH_OFF + MU_PRE_POLICY_HASH_LEN
== MU_PRE_LEN);
```

4. Frame Wire Format

4.1 Frame v2 Structure (4030 Bytes)

The complete proof unit is a 4030-byte binary frame transmitted as a single opaque blob:

```
Frame v2 = mu_pre (274 bytes) || T_E (1336 bytes) || sig (2420 bytes)
```

Component	Size	Description
mu_pre	274 bytes	Canonical payload (Section 3.3)
T_E	1336 bytes	Epoch public key
sig	2420 bytes	ML-DSA-44 signature

4.2 Epoch Public Key Layout (T_E, 1336 Bytes)

Offset	Bytes	Field
0	32	p (ExpandA seed)
32	1280	t _i (Power2Round high bits, d=13)
1312	8	epoch (uint64 LE)
1320	16	device_id16

4.3 Signature Layout (2420 Bytes)

Offset	Bytes	Component
0	2304	z
2304	84	h (hint vector)
2388	32	ĉ (challenge hash)

5. Epoch-Bound Verification Semantics

5.1 Static Credential Weakness

Static authentication artifacts create persistent attack surfaces. Compromised credentials remain valid across sessions, systems, and time horizons, substantially increasing replay risk. This is the primary structural failure of PKI-based IoT authentication.

5.2 Epoch Binding

AN-DNA bounds authentication validity to a specific epoch. The epoch value is embedded in both T_E and mu_pre, and the verifier enforces that they are identical

(Directive B). An authentication frame is valid only within the epoch for which its T_E was derived. Once the epoch registry advances, any frame carrying the prior T_E fails verification deterministically.

5.3 Prover-Side Epoch Mutation (D0 Bridge)

The mechanism by which a new T_E is derived from a device's evolving polynomial identity is defined by two domain separation constants in the contracts crate:

```
pub const EPOCH_SEED_DOMAIN: &[u8; 16] = b"ANDNA-EPOCH-SEED";
pub const MLDSA_SEED_DOMAIN: &[u8; 16] = b"ANDNA-MLDSA-SEED";
```

These constants anchor the prover-side D0 epoch-witness bridge at the verifier interface level. The D0 construction — which derives ML-DSA-44 key material from the epoch-mutating polynomial identity via domain-separated SHAKE256 — is specified in the companion protocol document and implemented on the prover side. The verifier requires only the resulting T_E and does not hold or reconstruct prover secrets.

5.4 Forward Epoch Isolation

Compromise of the signing key for epoch E does not expose the key for any prior epoch $E' < E$, because each epoch key is derived independently via the D0 bridge from a polynomial state that evolves through a HKDF-SHA512-derived pseudorandom walk. The verifier enforces epoch monotonicity at the registry level.

6. Public-Verifier Post-Quantum Architecture

6.1 Shared-State Verification Limitations

Traditional verification systems require centralized trust anchors, shared secrets, or verifier-held sensitive state. These architectures increase operational complexity, reduce portability, and centralize compromise risk. A verifier holding device secrets in an HSM is itself a high-value attack surface.

6.2 Public-Verifier Semantics

The vNext Phase 1 architecture removes the shared-state requirement. The verifier requires only:

- the canonical μ_{pre} payload
- the epoch public key T_E (public material only: p and t_1)
- the ML-DSA-44 signature

No device secrets, no genesis polynomial, and no HSM access are required at the verification boundary. T_E carries everything the verifier needs to reconstruct the ML-DSA-44 public key and validate the proof transcript.

6.3 ML-DSA-44 Implementation

The `andna-mlds44` crate implements signature verification via `liboqs`, the Open Quantum Safe library. This is an active implementation operating under FIPS 204 Category 2 parameters — not a conceptual alignment posture. ACVP sigver test vectors from the FIPS 204 test suite are validated in `crates/mlds44/tests/acvp_sigver.rs`.

Parameter	Value
q	8,380,417
n	256
k, ℓ	4, 4
d	13
η (eta)	2
τ (tau)	39
ω (omega)	80
γ_1	$2^{17} = 131,072$
γ_2	$(q - 1)/88 = 95,232$
β (beta)	$\tau \cdot \eta = 78$

Implementation Status Note

The Rust FFI backend (`VERIFY_ENGINE=rust`) performs full ML-DSA-44 verification via `liboqs`. The Python engine implements all structural directives (A, B, E) but uses a stub for the ML-DSA-44 step — it is designed for testing and development without a native build. Production deployments must use the Rust backend.

7. Verification Pipeline

7.1 Overview

All verification is performed by `verify_vnext()` in `andna-core`. The pipeline is deterministic, side-effect-free, and zero-allocation on the hot path. Failures are returned as typed error codes. μ is zeroized after use regardless of outcome.

7.2 Enumerated Verification Directives

Directive A — Structural Integrity of μ_{pre}

Before any cryptographic computation, the verifier checks that μ_{pre} carries the expected domain separator and version byte. A frame that fails Directive A is rejected before any lattice arithmetic is performed, preventing oracle exposure to malformed inputs:

- $\mu_{pre}[0x0040..0x0049]$ must equal "ANDNAAUTH" (9 bytes, no NUL, no hyphen)

- `mu_pre[0x0049]` must equal `0x01`

Directive B — Epoch Correlation

The epoch field in `mu_pre` must equal the epoch field in `T_E` (both uint64 little-endian). A mismatch indicates a "Frankenstein" payload and is rejected with `ERR_EPOCH_MISMATCH`:

```
mu_pre[0x006A..0x0072] == T_E[1312..1320]
```

Directive E — Device Identity Duality

The 32-byte device identifier in `mu_pre` must equal the SHAKE256 expansion of the 16-byte device UUID in `T_E`. A mismatch is rejected with `ERR_DEVICE_ID_MISMATCH`:

```
mu_pre[0x004A..0x006A] == SHAKE256(T_E[1320..1336], 32)
```

7.3 Full Pipeline Sequence

```
Step 0 (Directive A)  Validate domain_sep == "ANDNAAUTH" and version
                      == 0x01
Step 1                pk_hash binding: SHAKE256(T_E, 64) ==
mu_pre[0:64] [constant-time]
Step 2 (Directive B)  Epoch correlation: mu_pre.epoch == T_E.epoch
Step 3 (Directive E)  Device ID duality: mu_pre.device_id32 ==
SHAKE256(T_E.device_id16, 32)
Step 4                Compute  $\mu$  = SHAKE256(mu_pre, 64)
Step 5                Extract ( $\rho$ ,  $t_1$ ) from T_E
Step 6                ML-DSA-44 Verify( $\rho$  ||  $t_1$ ,  $\mu$ ,  $\sigma$ ) via liboqs
[Rust backend]
Step 7                Zeroize  $\mu$ 
```

7.4 Error Taxonomy

Code / Constant	Triggered By
0 / OK	Successful verification
1 / ERR_LENGTH	Frame != 4030 bytes
2 / ERR_MU_PRE	Directive A failure or <code>mu_pre</code> length
3 / ERR_TE	<code>T_E</code> length or malformed public key
4 / ERR_SIG	Signature length mismatch
5 / ERR_PK_HASH_MISMATCH	Step 1: pk_hash binding failure
6 / ERR_SIG_INVALID	ML-DSA-44 verification reject

7 / ERR_EPOCH_MISMATCH	Directive B: epoch correlation failure
8 / ERR_DEVICE_ID_MISMATCH	Directive E: device ID duality failure
100 / ERR_INTERNAL	liboqs engine error

8. Known Answer Test Vectors

Three KAT vectors are defined and validated byte-for-byte between the Rust crate and Python reference. These vectors anchor the canonical serialization and serve as regression guards for any future implementation. Regeneration: python3 tests/generate_transcript_kats.py

Vector	Input	First 8 Bytes of Output
KAT-T0	pk_hash = SHAKE256(zeros(1336), 64)	0f 32 d8 b3 56 85 27 04
KAT-T1	pk_hash = SHAKE256(patterned T_E, 64) ρ = 0x01..0x20, t_i = 0xAA×1280, epoch=5, id16=0xBB×16	f4 33 80 86 d8 c1 14 8d
KAT-T2	μ = SHAKE256(mu_pre_from_ T1, 64) Chains from KAT-T1 pk_hash	a5 08 ef bf 68 0d 8c 7f

KAT-T2 chains from KAT-T1: the pk_hash field of the mu_pre input is the T1 output, validating the full derivation path from T_E through to μ .

9. Verification Service Architecture

9.1 Dual-Backend Engine

The Python layer exposes a VerifyEngine protocol implemented by two backends, selected via the VERIFY_ENGINE environment variable:

Backend	Selection	ML-DSA-44 Enforcement
Python	VERIFY_ENGINE=python (default)	Stub — testing and development only
Rust FFI	VERIFY_ENGINE=rust	Real liboqs — required for production

9.2 Verifier Service Routes

Route	Method	Description
/verify/vnext	POST	Accept 4030-byte frame (octet-stream); returns ACCEPT/REJECT + run_id
/health	GET	Engine status and contract version
/evidence	GET	Export current verification evidence log as JSON

9.3 Evidence and Audit Log

Every verification event is captured as an append-only VerificationRecord, exportable as structured JSON via the /evidence endpoint. Given the frame_hash and contract_version, a clean machine can independently reproduce any verification outcome.

Field	Description
run_id	Timestamp-derived unique identifier
timestamp	ISO-8601 UTC
frame_hash	SHA-256 of the raw input frame
frame_len	Frame byte length
decision	"ACCEPT" or "REJECT"
error_code	Typed error code (0 = OK)
error_msg	Error description if rejected
engine	Backend that produced the result
contract_version	"vNext-Phase1-R1"
schema_version	"1.0.0"

10. Implementation Summary

10.1 Rust Workspace Crates

Crate	Role
andna-contracts	Single source of truth: constants, offsets, compile-time assertions
andna-codec	Zero-copy frame parsing; no crypto
andna-transcript	SHAKE256 hashing, pk_hash derivation, Directives B and E checks
andna-mlds44	ML-DSA-44 verify via liboqs; ACVP test vectors
andna-core	verify_vnext() orchestrator; pure verification, no I/O
andna-ffi	C ABI exports for Python ctypes integration
andna-ffi-cli	Command-line verification tool

10.2 Python Package Modules

Module	Role
contracts.py	Python mirror of Rust constants; offset and length definitions
frame_packer.py	build_mu_pre() and pack_frame_v2() frame assembly
engine.py	VerifyEngine protocol; Python and Rust backends
replay.py	Append-only ReplayLog and VerificationRecord
api.py	FastAPI verifier service
native.py	ctypes bindings to libandna_ffi
cli.py	Command-line interface

11. Security Scope and Non-Claims

11.1 What This Framework Proves

The AN-DNA verifier proves, for a given frame:

- the mu_pre payload was constructed canonically (Directive A)
- the payload is bound to the submitted T_E and no other (pk_hash binding check)
- the payload's epoch matches the T_E's epoch (Directive B)
- the device identity in the payload is derived from the T_E's hardware root (Directive E)
- the ML-DSA-44 signature over μ is valid under the epoch public key (Rust backend only)

11.2 What This Framework Does Not Prove

Non-Claim	Detail
Formal security proof	The security argument reduces to Module-LWE hardness in the random oracle model. A complete standard-model reduction with tight advantage bounds and explicit simulator construction remains future work.
FIPS validation	The liboqs ML-DSA-44 implementation is not ACVP-certified. Certification is planned.
Behavioral legitimacy	The verifier proves cryptographic consistency, not that the upstream data or requesting party has benign intent.
Genesis polynomial secrecy	Compromise of a device's genesis polynomial is catastrophic and requires

	hardware controls (TPM 2.0, TrustZone). The verifier cannot detect this.
Python ML-DSA-44	The Python engine stub always passes the signature step. It provides no ML-DSA-44 security guarantee.

11.3 Primary Operational Risks

- Prover-side compromise: an attacker with the genesis polynomial can derive all epoch keys for that device.
- Implementation drift: any language binding not derived from andna-contracts risks silent layout divergence.
- liboqs provenance: security depends on the correctness of the liboqs ML-DSA-44 implementation. Independent FIPS 204 validation is planned.
- Python stub in production: deploying the Python backend provides no ML-DSA-44 security guarantee.

12. Future Work

The following items represent the primary engineering and research requirements to advance this work toward a complete, externally reviewed specification:

1. Formal reduction proof. Complete the standard-model EUF-CTA reduction to Module-LWE hardness with explicit advantage bounds and simulator construction.
2. D0 bridge independent review. The deterministic epoch-witness bridge is a novel construction. External review by lattice cryptographers is required before production deployment.
3. ACVP-certified ML-DSA-44 backend. Replace the liboqs dependency with a FIPS 204 ACVP-certified implementation for deployments requiring formal validation.
4. Parameter scaling. The v3.3 shared-state legacy parameters ($n=256$, $q=3329$) yield a primal BDD work factor of approximately $2^{73} \cdot 6$ — below 128-bit classical security. This applies to the legacy shared-state architecture under active deprecation; vNext Phase 1 operates under FIPS 204 Category 2 ML-DSA-44 parameters.
5. Policy hash activation (Phase 2). The policy_hash field at offset 0x00F2 is allocated and offset-locked. Phase 2 will activate context-bound policy enforcement without requiring a layout change.
6. Rust verifier service. Replace the Python FastAPI service with a Rust-native Axum service for production-grade throughput.

13. Conclusion

AN-DNA provides a scoped, deterministic trust verification architecture. The core contribution of the vNext Phase 1 implementation is the transition from shared-state verification — which requires verifier-held device secrets in an HSM — to public-verifier

semantics, where the verifier holds only a public epoch key and validates an ML-DSA-44 Fiat-Shamir-with-aborts proof transcript.

The andna-core Rust workspace delivers:

- a single source of truth for all payload constants via compile-time assertions
- three cross-language KAT vectors validated byte-for-byte between Rust and Python
- three named verification directives (A, B, E) with typed error codes
- ML-DSA-44 verification via FIPS 204-aligned liboqs bindings (Rust backend)
- an append-only evidence log with structured audit records

The path to a submission-ready manuscript requires completing the formal reduction proof, obtaining independent D0 bridge review, and achieving an ACVP-certified ML-DSA-44 backend. These remain clearly delineated future work items.

The foundational principle remains intentionally constrained: a verification system should prove only what it can deterministically reproduce.

References

7. NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard. 2024.
8. NIST. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. 2024.
9. Lyubashevsky, V. Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures. ASIACRYPT 2009.
10. Open Quantum Safe Project. liboqs: Open-source C library for quantum-resistant cryptographic algorithms. <https://openquantumsafe.org>
11. Bernstein, D.J., Lange, T. Post-quantum cryptography. Nature, 2017.
12. Regev, O. On lattices, learning with errors, random linear codes, and cryptography. STOC 2005.
13. NIST SP 800-22. A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications. 2010.
14. Krawczyk, H., Eronen, P. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, 2010.
15. National Cybersecurity Center of Excellence. Migration to Post-Quantum Cryptography. NIST SP 1800-38, 2024.

Disclosures

AI Collaboration

Portions of this manuscript were developed in collaboration with AI tools, including Claude (Anthropic) and Gemini (Google). AI assistance was used in drafting, technical review, and structural analysis. All cryptographic claims, implementation decisions, and architectural choices are those of the author and have been independently verified against the reference codebase. Disclosed per COPE guidelines on AI use in research writing.

Independent Research

This work is independent research and is not affiliated with, endorsed by, or conducted on behalf of any employer, client, or institution.

Code Availability

The reference implementation (andna-core Rust workspace and Python verification layer) is available upon request. A public release is planned following independent security review of the D0 epoch-witness bridge construction.

Version and Currency

This manuscript reflects implementation state as of May 2026, vNext-Phase1-R1. It will be updated as formal proof work and external review are completed. Readers should not interpret any aspect of this document as a production security guarantee.